Context-Aware Product Recommendation with a Five-Agent TF-IDF Pipeline

Project Report | Reproducible local demonstration

Abstract

Context Cart is a locally executable demonstration of product recommendation from natural-language shopping queries and explicit preferences. Five Python agents parse a query, assemble request-scoped context, retrieve catalogue evidence, rank candidate products, and produce evidence-grounded explanations. Retrieval uses TF-IDF and metadata filtering; no LLM or vector database is implemented. A deterministic catalogue of 120 synthetic products supports an evaluation of keyword, TF-IDF, context-aware TF-IDF, and five-agent ranking. Results are limited to synthetic category labels and do not establish human preference quality or real-world performance.

1. Introduction

Retail search benefits from combining intent with explicit constraints such as budget, preferred brand, and channel. This project provides an inspectable and reproducible baseline, separating parsing, retrieval, ranking, and explanation.

2. Literature Review

Term weighting is a foundation of lexical text retrieval [1]. Recommender systems include content-based, collaborative, and hybrid approaches [2]. Retrieval-augmented generation combines retrieval and language generation [3]. This implementation performs retrieval-grounded recommendation but uses deterministic templates rather than an LLM.

3. Existing System and Limitations

The keyword baseline matches category strings but does not score textual relevance or user constraints. The prototype adds TF-IDF, context, ranking, and evidence display. It has no retailer feed, user history, dense embeddings, vector database, or external language model.

4. Proposed System and Contributions

The software contribution is an orchestrated five-agent implementation, traceable retrieval/ranking, budget feasibility handling, grounded explanation and comparison, plus reproducible metrics. No research novelty or performance improvement is claimed.

5. System Architecture

The Streamlit interface calls a central orchestrator. Five agents parse the query, carry explicit context, retrieve records, rank candidates, and explain results. The catalogue is validated and indexed in memory with scikit-learn TF-IDF. Seven editable Mermaid diagrams are in docs/diagrams/architecture.md.

6. Dataset Description

The local generator creates 120 deterministic synthetic products in six categories. Product attributes, brands, prices, ratings, stock, and review counts are fictional. All records state synthetic demonstration data. No restricted third-party dataset or customer information is used. See data/DATA_DICTIONARY.md and data/SOURCE.md.

7. Methodology

TF-IDF documents combine product name, category, brand, description, and specifications; English stop-word removal and 1–2 grams are used. Cosine similarity retrieves candidates. The ranking formula is 0.50 relevance + 0.20 rating + 0.15 budget + 0.10 availability + 0.05 explicit preference, which averages preferred-brand and selected-channel match. If any candidate meets a specified budget, over-budget candidates are filtered out. Explanations use catalogue evidence only.

8. Implementation

The Python package is under src/recommender and the Streamlit application is app.py. Input catalogue validation enforces required fields and numeric bounds. Customer context is request-scoped and not persisted. No paid API or cloud service is

required.

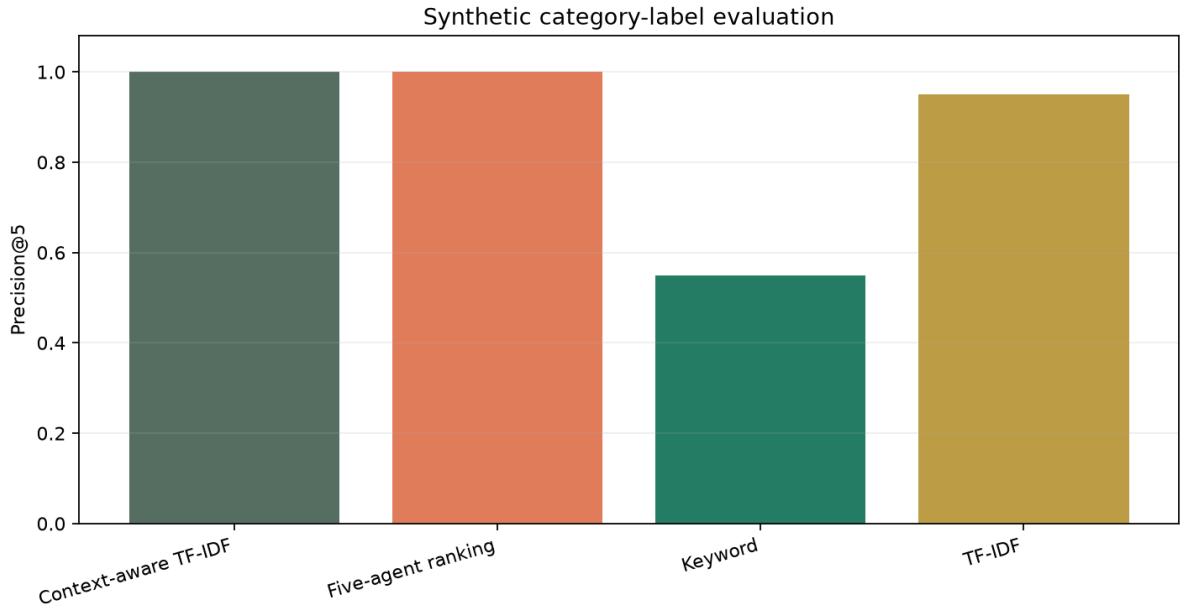
9. Experimental Setup

Four fixed query intents use expected category as the relevance label. Baselines are a category keyword filter, global TF-IDF, category-filtered TF-IDF, and five-agent ranking with budget. Metrics are Precision@5, Recall@5, MRR@5, NDCG@5, local single-process latency, and budget satisfaction. Category membership is a synthetic proxy, not a human judgment.

10. Results and Discussion

Method	P@5	R@5	MRR@5	NDCG@5	Latency ms	Budget
Context-aware TF-IDF	1.000	0.250	1.000	1.000	2.24	0.650
Five-agent ranking	1.000	0.250	1.000	1.000	12.23	1.000
Keyword	0.550	0.138	0.550	0.533	1.11	0.500
TF-IDF	0.950	0.237	1.000	0.967	2.24	0.600

The category-filtered methods score strongly because category is also the relevance definition. The figures therefore validate code behavior under a toy label, not shopper relevance. Latency is a single local run and is not a controlled benchmark.



11. Security, Privacy, Fairness, and Explainability

No external API or secret is required. Preferences are not persisted; explanations are constrained to retrieved fields. Fairness, accessibility, threat modeling, privacy certification, and penetration testing are not performed or claimed.

12. Conclusion and Future Scope

The project delivers a tested local workflow and measured synthetic evaluation. Further work needs licensed realistic catalogues, consented human relevance judgments, ranking ablations, fairness and accessibility evaluation, and operational hardening.

13. References

[1] G. Salton and C. Buckley, “Term-weighting approaches in automatic text retrieval,” Information Processing & Management, vol. 24, no. 5, pp. 513–523, 1988, doi: 10.1016/0306-4573(88)90021-0. [2] F. Ricci, L. Rokach, and B. Shapira, Eds., Recommender Systems Handbook, 2nd ed., Springer, 2015, doi: 10.1007/978-1-4899-7637-6. [3] P. Lewis et al., “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks,” NeurIPS, vol. 33, 2020.

14. Appendix

Tests cover catalogue schema/uniqueness, sample budget parsing and filtering, evidence-grounded explanations, explicit context, and finite metrics for all four baselines. Actual recommendation, product comparison, and context/evidence JSON are in outputs/examples/. A genuine browser screenshot was not available during artifact generation and is not fabricated. Full data dictionary and diagrams are maintained in the repository.